LLM INTERACTION ENGINEERING

(Course Code: VGULLM1SS)

Detailed Theory Notes for Examination

Unit 1: Introduction to LLM and Prompt Engineering

Unit 2: Prompt Engineering — Principles and Techniques

UNIT 1: INTRODUCTION TO LLM AND PROMPT ENGINEERING

Syllabus coverage: History and evolution of LLMs; Common NLP tasks; Generative models; Transformer, BERT and GPT; Tokenization and embedding; Fine-tuning vs. prompting; Role of the temperature parameter, top-k and top-p prompting; Popular LLMs and conversational models (GPT, Gemini, Llama, Mistral, Claude); Role of prompts in an AI model; Importance of effective prompts; Ethical considerations in prompt engineering.

1.1 Natural Language Processing (NLP) — Foundation

Natural Language Processing (NLP) is the branch of Artificial Intelligence concerned with enabling computers to read, understand, interpret, and generate human language — both text and speech — in a way that is useful and, ideally, humanlike. NLP sits at the intersection of computer science, linguistics, and machine learning, and every modern LLM is, at its core, a very large and powerful NLP system.

Why NLP is hard: human language is ambiguous (the same word can have multiple meanings depending on context), highly context-dependent (meaning shifts with surrounding sentences, tone, and culture), and irregular (idioms, sarcasm, grammar exceptions). Early NLP systems struggled precisely because they tried to handle this irregularity with fixed rules; later statistical and neural approaches instead let the system learn patterns of language directly from large volumes of real text.

Everyday applications built on NLP that you already use:

- Google Translate — machine translation
- Siri / Alexa — speech understanding (speech-to-text + language understanding)
- Gmail Smart Reply — text generation
- Product review filters / recommendation systems — sentiment analysis

Common NLP Tasks (with example prompts)

Task	Definition	Example
Translation	Converting text from one natural language to another while preserving meaning	"Translate this email into Hindi."
Summarization	Condensing a longer text into its most important points, either by extracting key sentences or generating a shorter paraphrase	"Summarize this article in 3 bullet points."
Question Answering (QA)	Providing a direct, specific answer to a factual or reasoning-based question, optionally grounded in a supplied document	"What year did India gain independence?"
Sentiment Analysis	Classifying the emotional tone of a piece of text — typically positive, negative, or neutral	"Is this product review positive or negative?"
Text Generation	Producing new, original, coherent text from a prompt or partial input	"Write a short poem about the monsoon."

These five tasks recur throughout the course because they are the standard benchmark tasks used to test and compare how well an LLM (or a given prompt) performs — you will use exactly these tasks in Experiment 1 of the lab component.

1.2 History and Evolution of LLMs

The development of LLMs did not happen overnight — it followed a clear historical arc, moving from hand-written rules, to statistical pattern learning, to neural networks that understand sequence, and finally to the transformer architecture that powers virtually every modern LLM. Understanding this arc explains why transformers were such a breakthrough and why today's models behave the way they do.

(a) Rule-Based Systems (1950s–1980s)

The earliest NLP systems used hand-written grammar rules and pattern-matching dictionaries. Every possible response had to be explicitly programmed by a human — the system had no ability to generalise beyond the rules it was given.

> **Example:** *ELIZA (1966) — an early chatbot that simulated a psychotherapist by matching keywords in the user's sentence to pre-scripted reply templates (e.g., turning "I am sad" into "Why do you say you are sad?").*

Limitation: rule-based systems break down immediately outside the narrow set of patterns their designers anticipated, and scaling them to handle the full richness of language is practically impossible.

(b) Statistical NLP & n-grams (1990s–2000s)

Instead of hard-coded rules, statistical NLP systems learned probabilities directly from large text corpora — essentially answering the question "which word is most likely to follow this one?" An n-gram model estimates the probability of a word based on the previous (n − 1) words that came before it.

> **Example:** *Early statistical machine translation systems (the pre-neural generation of Google Translate) worked this way — they translated phrase-by-phrase using probabilities learned from bilingual text pairs.*

Limitation: n-gram models can only remember a short, fixed window of previous words, so they cannot capture long-range context or meaning, and they suffer from data sparsity (many rare word combinations are never seen during training).

(c) The Neural Revolution — Word2Vec (2013)

Word2Vec introduced word embeddings: words are represented as vectors (lists of numbers) positioned in space so that words with similar meaning sit close together. This let machines capture meaning mathematically for the first time, based on the distributional hypothesis — the idea that words appearing in similar contexts tend to have similar meanings.

> **Famous analogy:** *king − man + woman ≈ queen — vector arithmetic on word embeddings captures real semantic and even grammatical relationships.*

Word2Vec is trained using one of two approaches: CBOW (Continuous Bag of Words), which predicts a target word from its surrounding context words, and Skip-gram, which does the reverse — predicting the surrounding context words from a single target word.

(d) Sequence Models — RNN / LSTM (2014–2016)

Recurrent Neural Networks (RNNs) read text one word (token) at a time while carrying forward a hidden "memory" state summarising everything read so far — a first serious attempt at handling context and sequence in language.

Problem with plain RNNs — the vanishing gradient problem: as a sequence gets longer, information from early tokens fades out (mathematically, gradients used to update the network shrink toward zero across many time steps), so the model effectively "forgets" the beginning of a long passage by the time it reaches the end.

LSTMs (Long Short-Term Memory networks) improved on this using gating mechanisms (input, forget, and output gates) that let the network deliberately decide what information to keep, discard, or output at each step — allowing longer, though still limited, memory of context.

(e) The Transformer Breakthrough (2017)

The paper "Attention Is All You Need" (Vaswani et al., Google, 2017) discarded recurrence entirely and introduced the transformer, built around a mechanism called self-attention.

- Self-attention: every word looks directly at every other word in the input at once and learns which ones matter most to it — no more reading strictly left to right, and no fixed-size memory bottleneck like an RNN's hidden state.
- Parallelization: because processing is not strictly sequential, training can happen massively in parallel on GPUs — this is what made scaling to huge models computationally feasible.
- Long-range context: a word at the end of a long passage can directly "attend to" a word at the very start, solving the RNN's forgetting problem.

Conceptually, self-attention works by converting every token into three vectors — a Query, a Key, and a Value. For each token, its Query vector is compared against every other token's Key vector to compute an attention score (how relevant that other token is), and the Value vectors are then combined using these scores as weights. Multi-head attention runs several of these attention computations in parallel (each "head" can learn to focus on a different kind of relationship, e.g., grammar vs. topic), and the results are combined. Because a transformer has no inherent sense of word order (unlike an RNN), positional encoding is added to each token's embedding to inject information about its position in the sequence.

(f) BERT and GPT (2018 onward)

The transformer architecture was specialised into two major families:

Aspect	BERT	GPT
Full form	Bidirectional Encoder Representations from Transformers	Generative Pre-trained Transformer
Maker / Year	Google, 2018	OpenAI, 2018 onward
Architecture	Encoder-only	Decoder-only

Reading direction	Reads the entire sentence at once (bidirectional) to build deep understanding	Reads left to right, predicting the next token one at a time (autoregressive)
Pre-training objective	Masked Language Modelling — randomly hides (masks) some tokens and trains the model to predict them from both directions, plus Next-Sentence Prediction	Causal / next-token prediction — trains the model to predict the next token given everything before it
Best at	Classification, fill-in-the-blank, search relevance, embeddings for downstream tasks	Writing, chatting, completing/generating open-ended text
Example	"The [MASK] sat on the mat" → predicts "cat"	"The cat sat on the" → predicts "mat"

(g) Scaling Up — The Modern LLM Era (2020s)

From 2018 onward, the same transformer idea was scaled up — more training data, more parameters (the model's internal learned weights), and more compute — giving rise to today's large language models such as the GPT-3/4 family, Gemini, Llama, Mistral, and Claude.

Key takeaway: the core architectural idea (the transformer, 2017) has not fundamentally changed since 2018 — what changed was scale. This scaling, combined with techniques such as instruction tuning and Reinforcement Learning from Human Feedback (RLHF) that align a raw pre-trained model with helpful, safe conversational behaviour, is what turned a research paper into consumer AI assistants used by hundreds of millions of people within a few years. Researchers also observed "emergent abilities" — capabilities (like multi-step reasoning) that appear only once a model crosses a certain scale threshold, and were largely absent in smaller models.

1.3 Generative Models

A generative LLM performs one core operation, repeatedly: it predicts the most likely NEXT token, given everything generated so far (this is called autoregressive generation). Internally, the model's final layer produces a score (logit) for every token in its vocabulary; a softmax function converts these scores into a probability distribution that sums to 1. The model then samples (or greedily picks the top-ranked) token from this distribution, appends it to the sequence, and repeats the entire process — one token at a time — to build a complete response.

> **Example:** *Given the input "The cat sat on the ___", the model computes a probability distribution over its entire vocabulary and is very likely to sample "mat" as the next token; it then repeats the process for the token after that, and so on until it produces an end-of-sequence signal or a length limit is reached.*

This simple mechanism — repeated next-token prediction — is the entire basis of everything an LLM can do: answering questions, writing code, translating, summarising, or holding a conversation are all, mechanically, very long chains of next-token predictions.

1.4 Popular LLMs and Conversational Models

Model	Maker	Weights	Typical Use

GPT (GPT-4 / GPT-5 family)	OpenAI	Closed	General-purpose assistant, coding, powers ChatGPT
Gemini	Google DeepMind	Closed	General assistant with deep integration into Google's products and search
Llama	Meta	Open	Research, self-hosting, and fine-tuning by third parties
Mistral	Mistral AI	Open / Closed mix	Efficient, smaller models suited to self-hosting
Claude	Anthropic	Closed	General assistant, writing, coding, with a strong emphasis on safety

"Open" weights mean the trained model parameters are published and can be downloaded, inspected, and fine-tuned by anyone (subject to licence terms); "closed" weights mean the model is only accessible through an API or product, and the underlying parameters are not released.

1.5 Tokenization

Before a model can process text, it must be split into tokens — small chunks, often smaller than a whole word (subwords). The model never sees a word like "unbelievable" as a single unit; it sees pieces and predicts/reads one piece at a time.

> **Example:** *"unbelievable" → un | believ | able (three tokens)*

Why subwords, not whole words? A finite, manageable token vocabulary (typically tens of thousands of entries) can still represent any word — even rare, misspelled, or previously unseen ones — by combining familiar sub-word pieces, instead of needing a separate vocabulary entry for every possible word form.

Common subword tokenization algorithms used in practice include:

- Byte-Pair Encoding (BPE) — starts with individual characters and iteratively merges the most frequently occurring pairs of symbols into new tokens, used by GPT-style models.
- WordPiece — similar to BPE but chooses merges that maximise the likelihood of the training data, used by BERT.
- SentencePiece — a tokenizer that treats the input as a raw stream of characters (including spaces), making it language-agnostic and useful for languages without clear word boundaries.

Tokenization also directly affects cost and context-window usage in real applications: API pricing and the model's maximum context length are both measured in tokens, not words or characters, which is why efficient prompt writing (Module 3) matters practically as well as stylistically.

1.6 Embeddings

Once text is tokenized, each token is converted into an embedding — a vector (a list of numbers, e.g. 768 or 4096 dimensions in real models) that captures something about its meaning. This lets the model work with numbers instead of raw text, and — as seen with Word2Vec — similar-meaning tokens end up with similar vectors (e.g., king/queen, happy/joyful).

> **Example:** *"believ" → [0.21, −0.08, 0.44, …, 0.13]*

Similarity between two embeddings is typically measured using cosine similarity (the cosine of the angle between two vectors) — the smaller the angle, the more semantically similar the tokens or pieces of text are considered to be. This is the mechanism underlying semantic search and Retrieval-Augmented Generation (RAG), which is covered in a later module.

An important distinction: static embeddings (like Word2Vec) give a word exactly one vector regardless of context, whereas contextual embeddings (produced inside transformer-based models like BERT and GPT) give the same word a different vector depending on the surrounding sentence — e.g., "bank" in "river bank" vs. "bank account" gets different representations.

1.7 Fine-Tuning vs. Prompting

These are the two fundamentally different ways to change or control a pre-trained model's behaviour for a specific use case.

Aspect	Fine-Tuning	Prompting
What changes	The model's WEIGHTS, via additional training on task-specific data	Only the INPUT text; the model's weights stay completely untouched
Requirements	Needs a labelled dataset + significant compute	No retraining needed — works immediately
Speed / Cost	Slower and more expensive to set up	Fast, cheap, immediate — can be changed on every single call
Result	A new, specialised version of the model	Steered behaviour on the same underlying model

Common variants of fine-tuning include full fine-tuning (updating all of a model's weights, which is expensive) and parameter-efficient fine-tuning methods such as LoRA (Low-Rank Adaptation), which freezes most of the model and trains only a small number of additional parameters — making fine-tuning far cheaper while retaining most of the benefit. Instruction tuning (training a base model on examples of instructions and desired responses) and RLHF (further refining a model using human preference feedback) are the techniques that turn a raw next-token predictor into a helpful conversational assistant.

When to use which?

Scenario	Better Fit
You need an answer in the next 5 minutes	Prompting
Narrow, repeated task at massive scale (e.g., classify 1M tickets in a fixed format)	Fine-tuning
You are exploring / iterating on what you even want	Prompting
Need a very specific, consistent voice across thousands of calls	Fine-tuning (or a strong system prompt)
No labelled training data or compute budget	Prompting

Since prompting requires no retraining, the prompt is the primary interface — the main lever available in most real-world applications to control model behaviour. This is exactly why prompt engineering (the subject of the

rest of this course) matters so much in practice: it is often far cheaper and faster than fine-tuning, and is the natural first thing to try.

1.8 Sampling Controls: Temperature, Top-k, Top-p

Recall from Section 1.3 that a generative model produces a probability distribution over its vocabulary at each step using softmax. Sampling controls determine exactly how the next token is chosen from that distribution.

Temperature — the "creativity knob"

Temperature (T) is a value used to rescale the logits before the softmax step, controlling how randomly the model samples from its predicted next-token probabilities.

- Low temperature (close to 0) → the probability distribution becomes sharply peaked around the single most likely token, so the model gives deterministic, focused, repeatable output.
- High temperature (above 1) → the distribution flattens out, giving lower-probability tokens a real chance of being picked, producing diverse, creative, less predictable output.

> **Example:** *Prompt: "Describe a rainy day in one sentence." At temperature 0.0 the wording stays nearly identical on repeated runs (safe, predictable phrasing); at 0.7 the phrasing varies but stays coherent; at 1.2 the output becomes unusual and can turn incoherent (e.g., introducing unrelated imagery like "neon reflections").*

Top-k Sampling

Restricts the model's choices to only the k tokens with the highest probability, then samples among just those — regardless of how high the temperature is set.

> **Example:** *top-k = 3 → the model can only pick from its 3 most probable next words, no matter what.*

Top-p (Nucleus) Sampling

Restricts sampling to the smallest possible set of tokens whose cumulative probability exceeds a threshold p (the "nucleus"). Unlike top-k, the size of this set can change dynamically from step to step depending on how confident the model is.

> **Example:** *top-p = 0.9 → keep adding the next most likely tokens, in order, until their combined probability passes 90%, then sample only from that set.*

In practice, temperature, top-k, and top-p are often combined, and are covered further as the "knobs" explored hands-on in Experiment 3 of the lab component.

1.9 Role of Prompts in an AI Model & Importance of Effective Prompts

A prompt is the input text used to steer a model's behaviour, without any retraining. Because prompting is the fastest and cheapest lever available (Section 1.7), the QUALITY of a prompt directly determines the quality, relevance, and usefulness of the output. An LLM has been described as "a genie with zero imagination" — it grants exactly what you wish for, not what you meant; a vague wish produces a weird or generic result, while a precise wish produces a targeted, useful one.

Prompt Type	Example & Result
Vague prompt	"Write about climate change." → Result: a generic, unfocused output with unclear length or audience — could go in any direction.
Well-specified prompt	"In 100 words, explain climate change to a 12-year-old, using one everyday analogy." → Result: scoped length, clear audience, concrete stylistic constraint — output is predictable and useful.

What made the better prompt better? Four ingredients (a preview of the five principles covered in full in Unit 2):

- Clarity — a specific, unambiguous task, leaving no room to guess intent.
- Context — enough background for the model to calibrate its answer (audience, purpose).
- Format — a stated output shape: length, structure, style.
- Constraints — explicit boundaries on what to include, exclude, or emphasise.

1.10 Ethical Considerations in Prompt Engineering

Concern	Description
Bias	Model outputs can reflect and amplify social, cultural, or demographic biases present in the training data, potentially producing unfair or stereotyped responses.
Hallucination	Fluent, confident-sounding text that is factually wrong — the model states an incorrect fact with the same tone of certainty as a correct one, which can mislead users who are not verifying claims.
Misuse	The same steering power that produces useful output can be used to generate misinformation, spam, or attempt to "jailbreak" (bypass) a model's built-in safety behaviour.
Responsible Use	Using AI tools transparently and appropriately — for example, disclosing AI assistance in academic or professional work, and not presenting AI-generated content as entirely one's own uncredited work.
Privacy	Prompts and outputs may inadvertently expose or request sensitive personal or confidential information; care must be taken with what data is included in a prompt.
Accountability	Because a model can produce plausible-sounding but wrong or biased output, a human should generally remain responsible for verifying and standing behind any decision or content ultimately produced with AI assistance.

1.11 Unit 1 — Key Terms Glossary

Term	Definition
NLP	AI field concerned with processing and understanding human language.
LLM	Large Language Model — a huge transformer-based model trained on massive text data.
Generative model	A model that predicts/generates the next token repeatedly to build output (autoregressive generation).
Self-attention	Mechanism letting every token weigh the relevance of every other token in the input.
Positional encoding	Information added to embeddings so a transformer knows token order.

Transformer	2017 neural network architecture using self-attention, replacing recurrence.
BERT	Encoder-only transformer, built for language understanding via masked-language modelling.
GPT	Decoder-only transformer, built for autoregressive language generation.
Tokenization	Splitting text into model-readable (often subword) pieces, e.g., via BPE.
Embedding	A numeric vector representation of a token's or text's meaning.
Fine-tuning	Retraining a model's weights on new/task-specific data.
LoRA	A parameter-efficient fine-tuning method that trains only a small added set of weights.
RLHF	Reinforcement Learning from Human Feedback — aligns a model with human preferences.
Prompt engineering	Designing input text to steer a model's output without retraining.
Temperature	A parameter controlling the randomness of token sampling.
Top-k / Top-p	Sampling methods that restrict the pool of candidate next tokens.
Bias	Training-data skew reflected/amplified in model output.
Hallucination	Confident but factually incorrect model output.

UNIT 2: PROMPT ENGINEERING — PRINCIPLES AND TECHNIQUES

Syllabus coverage: Designing effective prompts — the five principles of prompt engineering (Give Direction, Provide Examples, Specify Format, Evaluate Quality, Divide Labour); techniques and types of prompts (list generation, chunking, summarization); generating JSON and YAML; text style unbounding; Explain-It-Like-I'm-5; role prompting; GPT prompting tactics (avoiding hallucination by reference, thinking time, inner monologue, self-eval LLM responses); evaluating and validating prompts; classification with LLMs; meta-prompting; zero-shot and few-shot prompting.

2.1 Designing Effective Prompts: The Five Principles

An LLM has been described as "a genie with zero imagination" — it grants exactly what you wish for, not what you meant. A vague wish produces a weird or generic result; a precise wish produces a targeted, useful one.

> **Vague vs Precise:** *"Write about India." (vague — the model must guess audience, length, purpose, and angle) vs. "Write a 100-word paragraph on India's festivals, for a 10-year-old." (precise)*

The five principles of prompt engineering are:

1. Give Direction — Tell the model WHO it is (role), WHO it's talking to (audience), and WHAT you actually want (goal), including length and tone.
2. Provide Examples — Show, don't just tell. Give the model 1–2 examples of the desired output and let it copy the pattern (this is the seed idea behind zero-shot vs. few-shot prompting, covered in Section 2.9).
3. Specify Format — State the desired output shape explicitly: bullets, table, JSON, word/character limits, etc.
4. Evaluate Quality — Judge the output critically against a checklist rather than accepting it as-is (expanded in Section 2.6).
5. Divide Labour — Break a large or complex task into smaller, chained prompts, feeding the output of one step as the input to the next, rather than trying to do everything in one giant prompt.

Principle 1 — Give Direction (worked example)

> **Before:** *"Write about India."*
>
> **After:** *"You are a travel writer. Write a 100-word intro to India's festivals for first-time tourists, in a warm, upbeat tone."*

Ask yourself: What role? What audience? What length/tone? What is the purpose of the output? Notice that role + audience + length + tone were combined into a single line — direction does not need to be verbose to be effective, only complete.

Principle 2 — Provide Examples (worked example)

> **Prompt:** *"Turn each fact into a punchy tagline. Example: Fact: Coffee has caffeine. → Tagline: Wake up, one sip at a time. Now try: Fact: Solar panels convert sunlight to electricity."*

> **Output:** *"Sunshine in, power out."*

One good example often does more work than several extra sentences of instructions, because it demonstrates the exact tone, length, and structure expected, rather than describing them abstractly.

Principle 3 — Specify Format

Explicitly stating the required output shape (a numbered list, a table, a fixed set of JSON keys, a word count) removes ambiguity about how the answer should be structured, and is essential whenever the output will be read by downstream code rather than a human (see Sections 2.3 and 2.7).

Principle 4 — Evaluate Quality

Treat the model's first answer as a draft, not a final product. Actively check the output against clear criteria (Did it answer the actual question? Is it factually grounded? Is the format correct?) before using or trusting it — this mindset underlies the entire evaluation-and-validation checklist covered in Section 2.6.

Principle 5 — Divide Labour

Complex, multi-step tasks (e.g., "research a topic, then outline it, then write it, then edit it") are usually handled far more reliably by chaining several small, focused prompts together — each with a clear, narrow job — than by asking a single prompt to do everything at once. This is the same underlying idea used later in Prompt Chaining and LangChain pipelines (a later module).

2.2 Prompt Techniques and Types

List Generation

Ask for the exact count and exact structure the output should have — this stops the model from guessing how much is "enough", and prevents inconsistent list lengths across repeated calls.

> **Prompt:** *"List 5 pros and 5 cons of remote work, as two separate bulleted lists, no extra commentary."*

Chunking

LLMs have a fixed context window (a maximum number of tokens they can process at once); feeding a very large document in one go can cause the middle portion to get lost, truncated, or under-weighted. The fix: split the document into chunks, prompt each chunk separately, then stitch the results together.

> **Prompt:** *"Split the following report into ~500-word chunks, breaking only at paragraph boundaries. Number each chunk."*

In practical systems (covered further under RAG in a later module), chunks are often given a small overlap with the previous chunk so that information spanning a chunk boundary is not lost entirely from either side.

Summarization — Extractive vs. Abstractive

Type	Description & Example
Extractive	Pulls out the most important sentences, word-for-word, from the source. Safer for facts and figures since nothing is reworded. Example prompt: "Extract the 3 most important sentences from this article, unchanged."

Abstractive	Generates new sentences that capture the meaning, not copy-pasted from the source. Reads better and is more flexible on length, but carries a higher risk of subtly altering meaning. Example prompt: "Summarize this article in 3 bullet points, in your own words, for a non-expert reader."

Choosing between them depends on the downstream need: extractive summarisation is preferable when exact figures/quotes matter (e.g., legal or financial text); abstractive summarisation is preferable when readability and a consistent target length matter more than verbatim wording.

2.3 Generating Structured Output: JSON and YAML

Structured formats (JSON, YAML) allow downstream code to reliably parse the model's output automatically, which is essential whenever an LLM's output feeds into another program rather than being read directly by a person.

> **JSON prompt:** *"Extract the name, date, and amount from this receipt text. Return valid JSON only, with exactly these keys: name, date, amount. No extra text."*
>
> **JSON output:** *{ "name": "Aditi Sharma", "date": "2026-08-14", "amount": "₹1,240.00" }*

YAML is a lighter-weight, more human-readable alternative to JSON, often used for configuration-style output:

> **YAML prompt:** *"Extract the same details and return them as YAML, using the keys name, date, and amount."*

Common pitfalls when generating structured output:

- Inconsistent keys across repeated calls (e.g., "amount" vs. "total" on different runs).
- Trailing commas or malformed brackets that break a JSON parser.
- The model adding explanatory prose before or after the JSON/YAML block.

Fix: explicitly state "JSON only, no extra text" (or the YAML equivalent) and give the exact key schema up front — this combines Principle 3 (Specify Format) with Principle 2 (Provide Examples, i.e., show a sample of the exact desired structure).

2.4 Style Control: Style Unbounding & Explain-It-Like-I'm-5 (ELI5)

Technique	Description & Example
Style Unbounding	Same underlying facts, new register — the prompt changes tone/style, not content. Technical register: "Photosynthesis converts CO_2 and H_2O into glucose using light energy." Casual register: "Plants basically eat sunlight to cook up their own food."
Explain It Like I'm 5 (ELI5)	A specific, extreme case of style unbounding: push the model to the simplest possible register — short words, one idea per sentence, a familiar analogy. Example: "Explain what a black hole is, like I'm five years old." → "It's like a giant vacuum in space so strong that not even light can escape it."

These techniques are especially useful for adapting the same underlying content to different audiences (executives vs. engineers, experts vs. beginners) without having to re-derive the facts each time.

2.5 Role Prompting

Role prompting means assigning the model a persona or area of expertise. This can change the vocabulary, depth, and even the reasoning approach used in the answer — not just the surface tone.

> **Before:** *"Explain how a hash table resolves collisions."*
>
> **After:** *"You are a senior backend engineer explaining to a new intern. Explain how a hash table resolves collisions, using a real analogy."*

Ask yourself: Who is the expert here? Who is the audience? What tone or depth fits that role?

When Does a Role Actually Help vs. Just Decorate?

Helps When…	Just Decoration When…
Domain expertise changes the substance of the answer (legal, medical, engineering framing)	The role doesn't change what information is actually needed
The audience needs a specific tone or depth	The task is a purely factual lookup
You need a consistent persona maintained across a multi-turn chat	The role only adds words to the prompt without changing output quality

2.6 GPT Prompting Tactics (Reasoning-Reliability Tactics)

The following four tactics make a model's reasoning more reliable — not just its wording. They matter because a model can be fluent and confident while still being wrong (Section 1.10, Hallucination); these tactics are practical, prompt-level defences against that risk.

(a) Avoiding Hallucination by Reference (Grounding)

Ground the answer in a source text you supply, so the model derives its answer from that text rather than inventing one from memory. Reference grounding is considered the single biggest lever against confident-sounding wrong answers.

Approach	Example
Ungrounded	"What is the refund policy?" → the model guesses a plausible-sounding policy (e.g., "30-day returns, full refund") that may not match reality at all.
Grounded	"Answer only using the policy text below. If it's not stated, say 'not specified.' [paste policy text]" → the answer is traceable to the source, and honestly reports when information is missing rather than guessing.

(b) Thinking Time

Asking the model to reason step-by-step before answering improves accuracy on multi-step problems. This is the basic idea behind Chain-of-Thought (CoT) prompting, which is studied in far more depth in Module 5 (Advanced Prompt Engineering) — here it is introduced as a simple, immediately usable tactic.

> **Question:** *"A store has 3 shelves. Each shelf holds 8 boxes. Each box has 12 items. How many items total?"*

Without step-by-step thinking, the model may jump straight to an answer ("288" — or a rushed, wrong total for harder numbers). Prompting "Let's think step by step" instead produces: 3 shelves × 8 boxes = 24 boxes; 24

boxes × 12 items = 288 items; Answer: 288. Showing the intermediate work makes the same reasoning generalise to harder numbers, and makes errors easy to spot and correct.

(c) Inner Monologue

Separates the model's reasoning from its final answer — the user sees a clean final result, while the reasoning remains available separately (e.g., for debugging or auditing), typically by wrapping the reasoning inside explicit tags.

> **Prompt:** *"Put your reasoning inside <thinking></thinking> tags, then give the final answer after a line that says 'Answer:'"*
>
> **Output:** *<thinking> 3 shelves × 8 boxes = 24 boxes. 24 boxes × 12 items = 288 items. </thinking> Answer: 288*

(d) Self-Evaluation (Self-Eval) of LLM Responses

After the model answers, ask it to verify its own work — ideally by re-deriving the answer using a different method — before finalizing the response. This builds directly on Inner Monologue and Thinking Time.

> **Prompt:** *"Solve the problem. Then, in a <check> block, re-solve it using a different method. If the two disagree, flag it and give the corrected final answer."*

What self-eval catches vs. does not catch:

Catches	Does Not Catch
Arithmetic slips	Confidently wrong facts the model believes on both passes — this is rationalization, not true independent verification
Missed constraints	
Contradicted instructions	

This limitation is important: self-eval is a useful, cheap sanity check, but it is not a substitute for grounding (tactic a) or for independent human/external verification, because a model that is confidently wrong about a fact will typically remain confidently wrong when asked to double-check itself the same way.

2.7 Evaluating and Validating Prompts / Outputs

A practical checklist to run before trusting any LLM output — this operationalises Principle 4 (Evaluate Quality) from Section 2.1:

- Did it follow the requested format?
- Is it grounded — free of invented facts?
- Is it complete — nothing silently dropped?
- Is it consistent if you run it again?

Consistency is a validation question, not merely a temperature question — running the same prompt multiple times and comparing the outputs against this checklist is a standard, practical validation technique, and reveals whether a prompt's behaviour is stable enough to rely on in production. (Formal metrics for evaluating prompts

— accuracy, relevance, and coherence — are studied in depth in the next unit, Module 3: Prompt Evaluation and Fine Tuning.)

2.8 Classification with LLMs

LLMs can be used as classifiers via natural-language prompting, sorting free-text inputs into a fixed, predefined set of categories — without writing a traditional machine-learning classifier.

> **Prompt:** *"Classify this ticket into exactly one of: billing, technical, general. Respond with only the category name, nothing else."*
>
> **Output:** *billing*

Why output format matters for classification:

Works When…	Breaks When…
Downstream code parses the result automatically (a single, constrained label from a known set)	The model adds explanation, hedges, or invents a new label outside the given set

When you have a labelled set of test examples, classification accuracy can be measured directly as (number of correct predictions) ÷ (total predictions), and this is exactly what several of the course's lab experiments ask you to compute for zero-shot and few-shot classification prompts (see Section 2.9).

2.9 Meta-Prompting

Meta-prompting means using an LLM to critique or rewrite a prompt — i.e., prompts about prompts. This is the conceptual seed of the CLEAR framework's iterative refinement process, which is covered in the next unit (Module 3: Prompt Evaluation and Fine Tuning).

> **Prompt:** *"Here's a prompt I wrote: '[Your prompt]'. Identify anything ambiguous or missing, then rewrite it to be clearer."*

Meta-prompting is useful both for improving one's own prompts and, at a larger scale, is the underlying idea behind Automatic Prompt Engineering (APE), studied later in Module 5.

2.10 Zero-Shot and Few-Shot Prompting

These terms describe how many worked examples are supplied inside the prompt itself before asking the model to perform the task — this is also called in-context learning, since the model is "learning" the pattern purely from what is inside the current prompt, with no weight updates at all.

Type	Description & Example
Zero-shot prompting	The model is asked to perform a task with no examples at all — only an instruction. It relies purely on knowledge learned during pre-training. Example: "Classify this sentence as positive, negative, or neutral: 'The service was okay, nothing special.'" (no labelled examples given).
One-shot prompting	A single labelled example is given before the actual task, providing just enough demonstration of the expected format without a full set of examples.

Few-shot prompting	The prompt includes a small number of labelled examples (typically 2–5) demonstrating the input → output pattern before the actual task, helping the model infer the exact format and labels expected. Example: give 3 labelled sample support tickets (billing/technical/general) before asking it to classify a new, unseen ticket.

Trade-offs: zero-shot prompts are faster to write and use fewer tokens (cheaper and quicker), but can be less accurate or consistent on nuanced, ambiguous, or unusually formatted tasks. Few-shot prompts use more tokens — since the examples themselves consume context-window space and add cost — but generally improve accuracy, format-consistency, and label-adherence, and are especially useful for classification tasks where exact category names must be respected (Section 2.8). This directly connects back to Principle 2 (Provide Examples) from Section 2.1: few-shot prompting is that principle applied specifically and systematically to classification and pattern-following tasks.

2.11 Unit 2 — Key Terms Glossary

Term	Definition
Give Direction	Specifying role, audience, length, tone, and goal in a prompt.
Provide Examples	Including sample input/output pairs so the model copies the pattern.
Specify Format	Explicitly stating the desired output structure (bullets, table, JSON, etc.).
Evaluate Quality	Critically judging model output against a checklist rather than accepting it blindly.
Divide Labour	Breaking a complex task into smaller, chained prompts.
Chunking	Splitting a long document into smaller pieces before prompting, due to context-window limits.
Extractive summarization	Summary made of verbatim sentences pulled from the source.
Abstractive summarization	Summary written in new words that capture the source's meaning.
Style unbounding	Changing the tone/register of output while keeping the same facts.
ELI5	Explain It Like I'm 5 — simplest possible register with analogies.
Role prompting	Assigning the model a persona/expertise to shape its response.
Grounding	Restricting the model's answer to a supplied reference source to avoid hallucination.
Thinking time / CoT seed	Prompting the model to reason step-by-step before answering.
Inner monologue	Separating reasoning (hidden/tagged) from the final answer.
Self-eval	Having the model verify/re-derive its own answer before finalizing.
Classification prompting	Using an LLM to sort inputs into a fixed, predefined set of labels.
Meta-prompting	Using an LLM to critique or rewrite another prompt.
In-context learning	The model inferring a task pattern purely from examples given in the prompt, with no weight updates.
Zero-shot prompting	Performing a task with an instruction only, no examples.
One-shot prompting	Performing a task after being shown exactly one labeled example.
Few-shot prompting	Performing a task after being shown a few (2–5) labeled examples.